

I suggest this prompt structure below, it must be unified for all steps KO8G, KSC5, CKMLCP, ..., but every section Role, Context, Instructions, ... will be permanently stored in the config and will contain specific data for each step. Let's think about what is good in this prompt and what should be improved. Agent will send prompt with this unified structure to Llm, that will return response, I think it will as reasoning with included unified tool call format and append this response to prompt, agent will analyse the response and either run tool if response contain tool call format or send result to web user interface if Llm response contain final result format. After the tool has worked and returns the unified result agent append this result to prompt as section, for example "### Tool <tool_name> result" and send updated prompt to Llm. This cycle repeats until Llm return final result or max runs will reached. We should think about unified format that will append to prompt as section in cases when Llm return call tool (section call tool result) or just reasoning text then maybe it can be reasoning section append to prompt and send to Llm again.

The key idea that after every action prompt will expand with action result (Llm response, tool result, ...), that why prompt must have unified structure (I think sections) and unified format for tool result and Llm response, which will append to prompt as new sections and Llm every time will see the history, what has it done and what hasn't. However, I see one problem here, the size of Llm's context window is limited, and prompt can be silent cut, but if max_call will 7-10 and the Llm won't return large responses then it will fit in the context window.

Start prompt structure

Role

You are a SAP CO diagnostic assistant. # as example

Context

KO8G performs periodic settlement of CO internal orders for controlling area {{controlling_area}}, fiscal year {{fiscal_year}}, period {{period}}. After the settlement run, validation detected orders that were NOT settled due to errors: {{errors}}

Goal

Your task: diagnose the root cause for each failed order and provide clear resolution steps.

Tools

You have access to the following tools:

- Tool name - description ...
- Tool name - description ...

To call a tool, output:

```

<TOOL_CALL>
{
  "tool": "<TOOL_NAME>",
  "args": {
    "table": "<TABLE>",
    "fields": "<FIELD1>, <FIELD2>, ...",
    "where": "<SAP_WHERE_CLAUSE>",
    "max_rows": <N>
  }
}
</TOOL_CALL>

```

Instructions (or analysis_guidance) or maybe like that

Diagnostic Procedure (follow steps in order)

Step 1: Retrieve order master data

Use TOOL_READ_TABLE to read from ...

Step 2: Check order status (most common cause)

Evaluate the status fields for each order:

and so on

Analysis result

When you have diagnosed ALL errors, output:

<ANALYSIS_RESULT> # structure must be unified, but content will be different for different steps and saved in config.

We need think about how do this structure unified, the fields of this structure will used for display result table of errors in the web UI. I think it should be fields like error_object, type, status, error_code, error_msg, cause, recommendation, ... This fields may not fit on the screen, maybe we have to split to two tables errors and recommendations, like you show in USER_REPORT.

```

{
  "status": "completed", # I don't imagine for what it can used (what statuses can it have)
  "root_causes": [
    {
      "error_code": "<msg_num>",
      "error_msg": "<msg_text>",
      "cause": "<short cause description>",
      "error_objects": ["<AUFNR>", ...],
      "recommendation": "<fix description>",
    }
  ]
}
</ANALYSIS_RESULT>

```

the USER_REPORT I think we can drop because insted of it will used ANALYSIS_RESULT

CLAUDE

Отличный подход! Давай систематически разберём каждый аспект.

1. Оценка текущей структуры промпта

Что хорошо:

- Чёткое разделение на секции (Role, Context, Goal, Tools, Instructions, Result)
- Унифицированная структура для всех шагов (KO8G, KSC5, CKMLCP, ...)
- Идея с расширяющимся промптом через секции — это по сути реализация **ReAct pattern** (Reasoning + Acting)
- Конфигурируемость — каждая секция хранится отдельно и подставляется

Что нужно улучшить:

Проблема 1: Нет секции Rules/Constraints

Нужна отдельная секция с жёсткими правилами поведения, которые модель не должна нарушать.

Проблема 2: Формат промежуточных секций не определён

Ты правильно заметил — нужен унифицированный формат для секций, которые добавляются в ходе работы.

Проблема 3: ANALYSIS_RESULT нужно доработать для UI

Проблема 4: Нет механизма для ситуации "не могу определить причину"

2. Доработанная структура

```
yaml
```

```
# =====  
# UNIFIED PROMPT TEMPLATE  
# =====  
# Sections marked [STATIC] are loaded from config once  
# Sections marked [DYNAMIC] are appended during agent loop  
# =====
```

```
prompt_template: |
```

```
## Role [STATIC]  
{{role}}  
  
## Context [STATIC]  
{{context}}  
  
## Goal [STATIC]  
{{goal}}  
  
## Tools [STATIC]  
{{tools}}  
  
## Instructions [STATIC]  
{{instructions}}  
  
## Output Format [STATIC]
```

```
{{output_format}}
```

```
## Rules [STATIC]
{{rules}}
```

```
## Execution Log [DYNAMIC]
{{execution_log}}
```

Почему именно так:

- **Role, Context, Goal** — фрейминг задачи
- **Tools** — что доступно
- **Instructions** — как действовать (decision tree)
- **Output Format** — как оформить финальный ответ
- **Rules** — ограничения и constraints
- **Execution Log** — растущая история действий

3. Секция Rules

```
yaml
```

```
rules: |
  ## Rules
  1. Follow Instructions steps sequentially. Do NOT skip steps.
  2. Make ONE tool call at a time. STOP after <TOOL_CALL> and wait
    for the result.
  3. Do NOT fabricate data. If a tool returns no data or an error,
    report it honestly.
  4. If you cannot determine the root cause after all available
    checks, set cause to "Undetermined" and provide manual
    investigation steps.
  5. Respond in the same language as the Context section.
  6. Do NOT repeat previous tool calls with the same arguments.
    If you already have the data in Execution Log, use it.
  7. Keep your reasoning concise. Focus on diagnostics,
    not explanations of SAP theory.
  8. When you have results for ALL error objects, output
    <ANALYSIS_RESULT> immediately. Do not make unnecessary
    additional tool calls.
```

4. Execution Log — ключевая идея

Это самая важная часть. Вместо отдельных тегов для разных типов секций, предлагаю **единый формат записи** в Execution Log:

```
yaml
```

```
execution_log_format: |
  ## Execution Log

  ### Action 1: LLM Reasoning
  <REASONING>
```

```
Parsing errors from context. Found 3 failed orders:
- [jg:x5tabel_8]: error KD205
- FK01: error KD205
- FK02: error KD205
All share the same error code. Will query AUFK first.
</REASONING>

### Action 2: Tool Call
<TOOL_CALL>
{"tool": "sap_read_table", "args": {"table": "AUFK", ...}}
</TOOL_CALL>

### Action 3: Tool Result
<TOOL_RESULT>
{"tool": "sap_read_table", "table": "AUFK", "status": "ok",
 "count": 3, "data": [...]}
</TOOL_RESULT>

### Action 4: LLM Reasoning
<REASONING>
AUFK data received. All 3 orders exist, KOKRS matches.
LOEKZ is empty. Now checking statuses in JEST...
</REASONING>

### Action 5: Tool Call
<TOOL_CALL>
{"tool": "sap_read_table", "args": {"table": "JEST", ...}}
</TOOL_CALL>

### Action 6: Tool Result
<TOOL_RESULT>
{"tool": "sap_read_table", "table": "JEST", "status": "ok",
 "count": 12, "data": [...]}
</TOOL_RESULT>

### Action 7: LLM Reasoning + Final Result
<REASONING>
Status check: all orders have I0002 active, no blocking statuses.
Status is not the issue. Error is KD205 = missing settlement rule.
Checking COBRB...
</REASONING>

... и так далее до ...

### Action N: Final Result
<ANALYSIS_RESULT>
{...}
</ANALYSIS_RESULT>
```

Формат для агента — правила парсинга:

Ответ LLM содержит → Действие агента ————— <TOOL_CALL>
 {...} → Выполнить tool Добавить в лог: "Action N: Tool Call" + содержимое "Action N+1: Tool Result" + результат Отправить обновлённый промпт в LLM <ANALYSIS_RESULT>{...} → Парсить JSON Отправить в Web UI ЗАВЕРШИТЬ Нет тегов (только текст)→ Добавить как "Action N: LLM Reasoning" Отправить обновлённый промпт в LLM (модель "задумалась", нужен ещё цикл)

Нумерация:

Агент автоматически нумерует Action N+1 при каждом добавлении. LLM видит полную историю и понимает, где остановилась.

5. ANALYSIS_RESULT — унифицированная структура

Для Web UI тебе нужно два списка: ошибки (детально по объектам) и рекомендации (сгруппированные). Вот унифицированная структура:

yaml

```
analysis_result_format: |
  <ANALYSIS_RESULT>
  {
    "status": "completed|partial|undetermined",
    "errors": [
      {
        "object_type": "ORDER|COST_CENTER|MATERIAL|...",
        "object_id": "FK01",
        "object_desc": "Заказ производства FK01",
        "error_code": "KD205",
        "error_msg": "Maintain the settlement rule of the sender",
        "cause": "Settlement rule is missing for this order",
        "cause_details": "No records found in COBRB for OBJNR ORFK01",
        "resolution_id": 1
      },
      {
        "object_type": "ORDER",
        "object_id": "FK02",
        "object_desc": "Заказ производства FK02",
        "error_code": "KD205",
        "error_msg": "Maintain the settlement rule of the sender",
        "cause": "Settlement rule is missing for this order",
        "cause_details": "No records found in COBRB for OBJNR ORFK02",
        "resolution_id": 1
      }
    ],
    "resolutions": [
      {
        "id": 1,
        "title": "Create missing settlement rules",
        "steps": [
          "Open transaction K002",
          "Enter order number",
          "Go to Settlement Rule tab",
          "Maintain receiver (cost center, GL account, etc.)",
          "Set distribution percentage to 100%",
        ]
      }
    ]
  }
```

```
      "Save"
    ],
    "affected_objects": ["FK01", "FK02"],
    "sap_transaction": "K002"
  }
]
}
</ANALYSIS_RESULT>
```

Поле `status` — вот зачем оно нужно:

Status	Когда	Что делает UI
complete d	Все ошибки диагностированы	Показывает полный отчёт
partial	Часть ошибок диагностирована, часть нет	Показывает отчёт + предупреждение
undetermi ned	Не удалось определить ни одной причины	Показывает сырые данные + "обратитесь к консультанту"

Разделение `errors` и `resolutions` через `resolution_id`:

Это решает проблему с UI:

Таблица 1 — Ошибки:

#	Объект	Описание	Ошибк а	Причина	Решение →
1	FK01	Заказ производства FK01	KD205	Нет правила расчёта	#1
2	FK02	Заказ производства FK02	KD205	Нет правила расчёта	#1
3	[jg:x5tabel_9]	Технический заказ для RU06	KD205	Нет правила расчёта	#1

Таблица 2 — Рекомендации:

#	Действие	Транзакция	Заказы
1	Создать правило расчёта в K002...	K002	FK01, FK02, [jg:x5tabel_10]

Не дублируем одну и ту же рекомендацию для каждого заказа.

6. Полный конфиг для K08G

```
yaml

# =====
# Config: K08G – CO Internal Order Settlement
# =====
```

```
step_id: K08G
step_name: "CO Internal Order Settlement"
```

```
role: |
```

```
You are a SAP CO diagnostic assistant specializing in
internal order settlement analysis.
```

```
context_template: |
```

```
Transaction K08G performs periodic settlement of CO internal
orders for controlling area {{controlling_area}}, fiscal year
{{fiscal_year}}, period {{period}}.
After the settlement run, validation detected orders that were
NOT settled due to errors:
{{errors}}
```

```
goal: |
```

```
Diagnose the root cause for each failed order listed in the
Context and provide actionable resolution steps. Every error
object must be accounted for in the final result.
```

```
tools: |
```

```
You have access to the following tools:
```

```
### sap_read_table
```

```
Reads data from SAP database tables via RFC_READ_TABLE.
```

```
To call:
```

```
<TOOL_CALL>
```

```
{
```

```
  "tool": "sap_read_table",
```

```
  "args": {
```

```
    "table": "<TABLE_NAME>",
```

```
    "fields": "<FIELD1>, <FIELD2>, ...",
```

```
    "where": "<SAP_WHERE_CLAUSE>",
```

```
    "max_rows": <number, default 100>
```

```
  }
```

```
}
```

```
</TOOL_CALL>
```

```
Where clause syntax:
```

- Equals: FIELD EQ 'VALUE'
- In list: FIELD IN ('VAL1', 'VAL2', 'VAL3')
- Combined: FIELD1 EQ 'X' AND FIELD2 IN ('A', 'B')

```
instructions: |
```

```
## Diagnostic Procedure (follow steps in order)
```

```
### Step 0: Parse error list
```

```
Extract from Context errors:
```

- Object ID: from "sender" field – token after "ORD ",
before description text.

```
Example: "ORD FK01 Заказ производства" → "FK01"
```


- Error code: msg_num
- Error text: message

Group by error code.

Step 1: Retrieve order master data

Read table AUFK.

Fields: AUFNR, AUART, AUTYP, BUKRS, KOKRS, WERKS, LOEKZ

Where: AUFNR IN (<extracted order numbers>)

AUFNR formatting:

- Digits only → pad to 12 chars: "5000000" → "000005000000"
- Has letters → as-is: "FK01" → "FK01"

Checks:

- No rows → order does not exist
- KOKRS ≠ {{controlling_area}} → wrong controlling area
- LOEKZ = 'X' → deletion flag set

Step 2: Check order status via JEST

Read table JEST.

Fields: OBJNR, STAT, INACT

Where: OBJNR IN (<"OR" + AUFNR from Step 1 results>)

Interpretation:

- Only rows where INACT is empty = active status
- INACT = "X" → status inactive, ignore
- "I" prefix = system status, "E" prefix = user status

System status decision:

Active STAT	Status	Can settle?	Fix
I0002 (no blockers)	REL	✅	–
I0001 without I0002	CRTD	❌	Release in K002
I0003	TECO	❌	Revoke TECO in K002
I0045	CLSD	❌	Reopen in K002
I0046	LKD	❌	Unlock in K002
I0076	DLFL	❌	Remove flag or exclude

Check priority: I0076 > I0045 > I0003 > I0046 > I0001

If blocking status found → this is the root cause.

Active E-statuses: note as potentially blocking,
recommend checking in K003.

Informational (ignore): I0016, I0118, I0215, I0420, I0485

Step 3: Error-specific checks

KD205 "Maintain the settlement rule of the sender"

Read table COBRB.

Fields: OBJNR, BURESSION, KONTY, EMPGE, PROZS

Where: OBJNR IN (<from Step 2>)

- No rows → rule missing → create in K002

- Sum PROZS $\neq$ 100 $\rightarrow$ rule incomplete $\rightarrow$ fix in K002
- Sum = 100 $\rightarrow$ receiver may be invalid $\rightarrow$ note EMPGE, KONTY

KD201 "No costs to settle"

No unsettled costs. Not a real error. Exclude from run.

KD210 "Settlement period locked"

Check period lock in OKP1.

Step 4: Output final result

When ALL error objects are diagnosed, output <ANALYSIS_RESULT>.

output_format: |

Output Format

When all errors are diagnosed, output exactly:

<ANALYSIS_RESULT>

{

 "status": "completed|partial|undetermined",

 "errors": [

 {

 "object_type": "<ORDER|COST_CENTER|...>",

 "object_id": "<ID>",

 "object_desc": "<description from error sender>",

 "error_code": "<msg_num>",

 "error_msg": "<message text>",

 "cause": "<root cause description>",

 "cause_details": "<what diagnostic data confirmed this>",

 "resolution_id": <N>

 }

],

 "resolutions": [

 {

 "id": <N>,

 "title": "<action title>",

 "steps": ["<step1>", "<step2>", ...],

 "affected_objects": ["<ID1>", "<ID2>"],

 "sap_transaction": "<tcode>"

 }

]

}

</ANALYSIS_RESULT>

Status values:

- "completed": all errors diagnosed
- "partial": some errors diagnosed, some not
- "undetermined": could not determine any root cause

rules: |

Rules

1. Follow steps sequentially. Do not skip.
2. One <TOOL_CALL> at a time. Stop and wait after each call.

3. Do not invent data. Report unknowns honestly.
4. Do not repeat a tool call with identical arguments.
Use data from Execution Log.
5. If root cause cannot be determined, set cause to "Undetermined – manual investigation required" and provide what to check manually.
6. Respond in the language of the Context section.
7. Keep reasoning concise and focused on diagnostics.
8. Every error object from Context must appear in the final ANALYSIS_RESULT errors array.
9. Maximum tool calls: {{max_tool_calls}}. If you reach this limit, output <ANALYSIS_RESULT> with status "partial" for unresolved items.

max_tool_calls: 8

7. Алгоритм агента (псевдокод)

python

```
def run_analysis(step_config, errors, params):
    # 1. Build initial prompt
    prompt = build_prompt(step_config, errors, params)
    execution_log = []
    action_counter = 0

    for iteration in range(step_config.max_tool_calls + 3):
        # 2. Call LLM
        response = llm.call(prompt)
        action_counter += 1

        # 3. Parse response
        if has_tag(response, "ANALYSIS_RESULT"):
            # Final result
            result_json = extract_tag(response, "ANALYSIS_RESULT")
            result = parse_json(result_json)
            validate_result(result, errors) # check all objects covered
            return result

        elif has_tag(response, "TOOL_CALL"):
            # Tool call + possible reasoning before it
            reasoning = extract_text_before_tag(response, "TOOL_CALL")
            tool_call = extract_tag(response, "TOOL_CALL")
            tool_args = parse_json(tool_call)

            # Log reasoning if present
            if reasoning.strip():
                action_counter += 1
                execution_log.append({
                    "action": action_counter,
                    "type": "reasoning",
                    "content": reasoning.strip()
                })
```

```
    })

    # Log tool call
    action_counter += 1
    execution_log.append({
        "action": action_counter,
        "type": "tool_call",
        "content": tool_call
    })

    # Execute tool
    tool_result = execute_tool(tool_args)

    # Log tool result
    action_counter += 1
    execution_log.append({
        "action": action_counter,
        "type": "tool_result",
        "content": json.dumps(tool_result)
    })

    # Rebuild prompt with updated log
    prompt = build_prompt(
        step_config, errors, params,
        execution_log
    )

else:
    # Pure reasoning, no action
    action_counter += 1
    execution_log.append({
        "action": action_counter,
        "type": "reasoning",
        "content": response.strip()
    })
    prompt = build_prompt(
        step_config, errors, params,
        execution_log
    )

# Max iterations reached
return {
    "status": "partial",
    "errors": [],
    "resolutions": [],
    "_note": "Max iterations reached"
}
```

```
def build_prompt(config, errors, params, execution_log=None):
    prompt = f"""## Role
{config.role}
```

```
## Context
{config.context_template.format(**params, errors=errors)}

## Goal
{config.goal}

## Tools
{config.tools}

## Instructions
{config.instructions}

## Output Format
{config.output_format}

## Rules
{config.rules}
"""

if execution_log:
    prompt += "\n## Execution Log\n"
    for entry in execution_log:
        type_label = {
            "reasoning": "LLM Reasoning",
            "tool_call": "Tool Call",
            "tool_result": "Tool Result"
        }[entry["type"]]

        tag = {
            "reasoning": "REASONING",
            "tool_call": "TOOL_CALL",
            "tool_result": "TOOL_RESULT"
        }[entry["type"]]

        prompt += f"\n### Action {entry['action']}: "
        prompt += f"{type_label}\n"
        prompt += f"<{tag}>\n{entry['content']}\n</{tag}>\n"

    prompt += "\n### Continue from here\n"

return prompt
```

8. Оценка контекстного окна

Прикинем размер промпта:

Компонент	Примерный размер (tokens)
Static sections (Role, Context, Goal, Tools, Instructions, Format, Rules)	~2000-2500

Компонент	Примерный размер (tokens)
Errors input (10 ошибок)	~300-500
Per iteration (reasoning + tool call + tool result)	~300-800
8 iterations worst case	~2400-6400
Total worst case	~5000-9000

Для Claude (200K), GPT-4o (128K), или даже моделей с 32K — это комфортно.

Но если ошибок будет 100+ и tool results будут большими, стоит добавить:

```
python

# В build_prompt — сжатие старых записей лога
if total_tokens(execution_log) > TOKEN_LIMIT:
    # Сохраняем последние 3 action, остальные суммаризируем
    old_entries = execution_log[:-3]
    summary = f"Previous actions (1-{old_entries[-1]['action']}): "
    summary += "Queried AUFK (3 orders found), JEST (statuses OK)"
    execution_log = [{"action": 0, "type": "reasoning",
                        "content": summary}] + execution_log[-3:]
```

9. Главные решения

Решение	Обоснование
Единый Execution Log вместо отдельных секций	Проще для агента и LLM, чёткая хронология
Теги <TOOL_CALL> , <TOOL_RESULT> , <REASONING> , <ANALYSIS_RESULT>	Надёжный парсинг, модели хорошо следуют XML-тегам
errors[] + resolutions[] с resolution_id	Нет дублирования рекомендаций, удобно для UI
status: completed/partial/undetermined	UI знает как отображать результат
"### Continue from here" в конце лога	Явный сигнал модели продолжать
Максимум tool calls в Rules	Модель сама знает лимит и может спланировать